Agenda:

1. Nearest Smaller & Greater Element on Left
2. Nearest Smaller & Greater Element on Right
3. Largest Rectangle in a Histogram

Q1: Given an integer array A[ ], find the **index** of **nearest smaller element** on left for all i index in A[ ].

Formally, for all **i** find **j** such that **A[j] < A[i]**, **j < i** and **j is maximum**.

If there is **no smaller element** on left side, ans is **-1** for that index.

Example: A[ ] = [8, 2, 4, 9, 7, 5, 3, 10]

0 1 2 3 4 5 6 7

Answer = [-1, -1, 1, 2, 2, 2, 1, 6]

Bruteforce → ∀ i, travel from (i-1) to 0, first element < A[i] is nearest smaller to A[i].

TC = O(N²) SC = O(1)

Solution

A = [8 2 4 9 7 3]

0 1 2 3 4 5

ans → -1 -1

A[1] < A[0], A[1] < A[0] < A[i] ⇒ A[1] < A[i]

if 0 is ans for any index i > 1

⇒ A[0] < A[i]

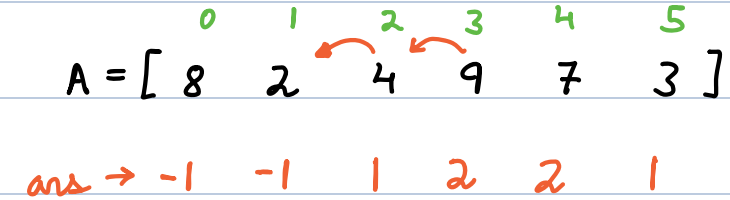
⇒ ∀ j → 1 to (i-1) A[j] ≥ A[i] ✗

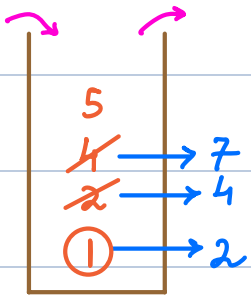
A = [_ _ 8 _ _ 5 _ _ _]

i

if 8 < A[i] ⇒ 5 < A[i] & index of 5 > index of 8

⇒ index of 8 cannot be answer.

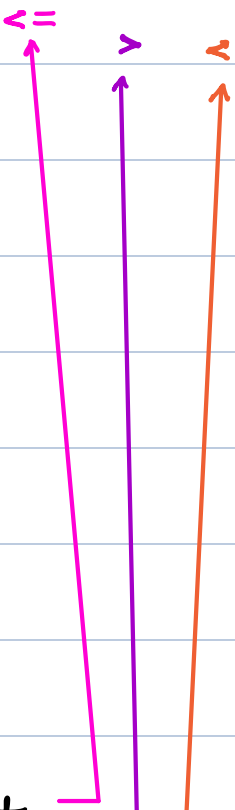





```
// st → empty stack
for i → 0 to (N-1) {
    while (!st.isEmpty() && A[st.peek()] >= A[i]) {
        st.pop()
    }
    if (st.isEmpty())      ans[i] = -1
    else                   ans[i] = st.peek()
    st.push(i)
}
return ans
```

TC = O(N)

SC = O(N)



Q2 → Find index of nearest greater element on left.

Q3 → Find index of nearest less than equal to on left.

Q4 → Find index of nearest greater than equal to on left.

Q5: Given an integer array A[ ], find the **index** of nearest smaller element on **right** for all i index in A[ ].

Formally, for all **i** find **j** such that **A[j] < A[i]**, **j > i** and **j is minimum**.

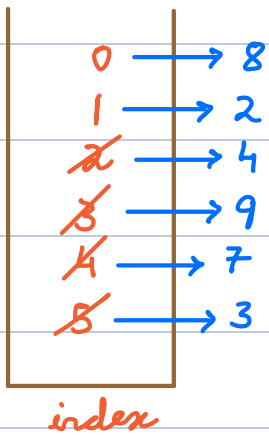
If there is **no smaller element** on right side, ans is **-1** for that index.

Example: A = [4, 8, 5, 2, 25]

Answer = [3, 2, 3, -1, -1]

```
          0  1  2  3  4
A = [4, 8, 5, 2, 25]
ans → 3  2  3 -1 -1
```

```
          0  1  2  3  4  5
A = [ 8   2  4  9  7  3 ]
ans → 1  -1  5  4  5 -1
```




```
// st → empty stack
for i → (N-1) to 0 {
    while ( !st.isEmpty() && A[st.peek()] >= A[i])
        st.pop()
    if (st.isEmpty()) ans[i] = -1
    else ans[i] = st.peek()
    st.push(i)
}
return ans
```

TC = O(N) SC = O(N)

Q6 → Find index of nearest greater element on right.

Q7 → Find index of nearest less than equal to on right.

Q8 → Find index of nearest greater than equal to on right.

Q → ∀i, find index of nearest smaller on left.

SC = O(1)

```
          0   1   2   3   4   5
A = [ 8   2   4   9   7   3 ]

ans → [-1  -1   1   2   2   1 ]
```

```
          0   1   2   3   4   5   6
A = [ 5   7   12  6   8   3   2 ]

ans = [ -1  0   1   0   3   -1  -1 ]
```

if A[3] > A[i] // i > 3

→ A[1], A[2] > A[i] ∵ ans[3] = 0

```
for i → 0 to (N-1) {
    j = i-1
    while (j != -1 && A[j] >= A[i])  j = ans[j]
    ans[i] = j
}
return ans
```

TC = O(N) SC = O(1)

Real Life Scenario

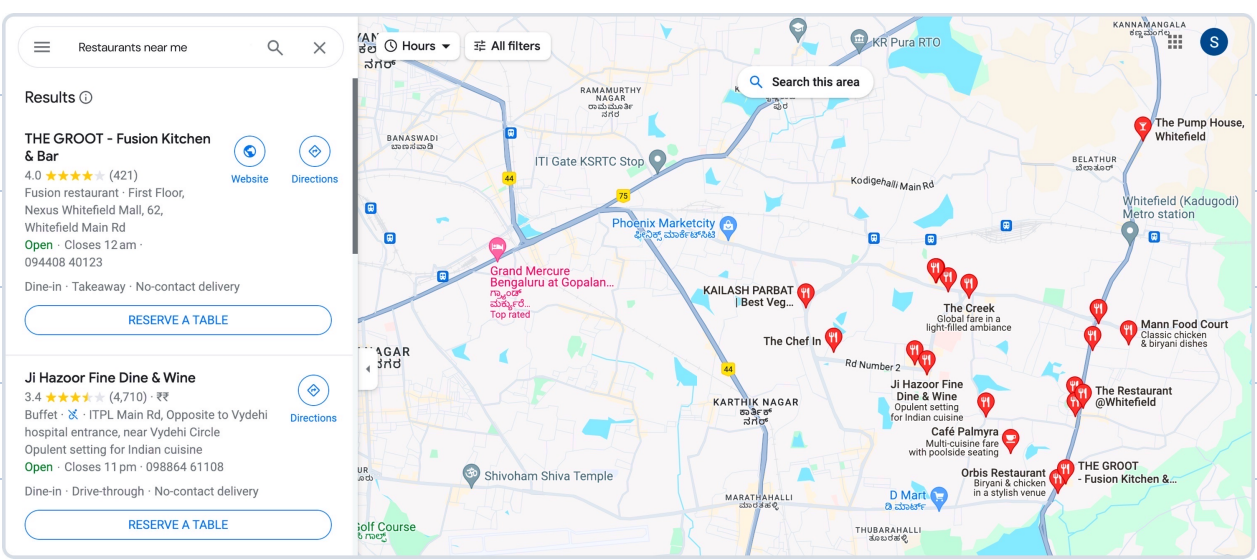
- **A person uses Google Maps to find the nearest restaurants and picks one based on it's proximity. Unfortunately, after visiting, they realised that the restaurant didn't meet their expectations.**

Task

You have a list of restaurants and their ratings. For each restaurant, we're going to find the next restaurant to the right on the list that's not just close but also has a higher rating than the current one. If there's no better option on the list, we'll say there's none available.

Problem

Given a sequence of restaurants listed on Google Maps with their ratings, create a tool that helps users discover the rating of the next higher-rated restaurant to the right for each listed establishment.

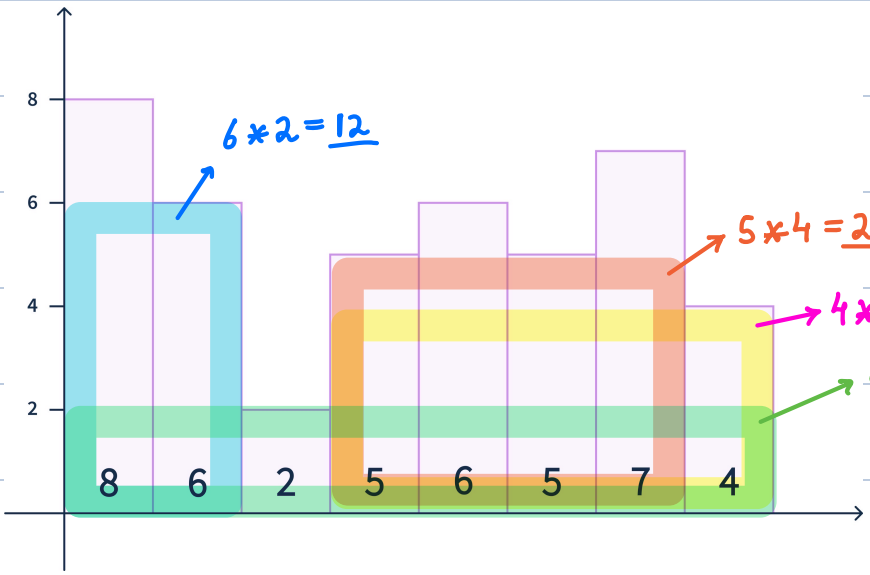


Q9: Given an integer array **A[ ]**, where **A[i]** = height of i-th bar. **Width** of each bar is = **1**.

Find the **area** of the largest rectangle formed by continuous bars.

height * width

Example: A = [8, 6, 2, 5, 6, 5, 7, 4]



Ans = 20

Bruteforce → ∀i,j → H = find min A[i—j]

W = j - i + 1 , calculate area & keep max.

```
ans = 0
for i → 0 to (N-1) {
    H = A[i]
    for j → i to (N-1) {   // i — j
        H = min (A[j], H)
        area = H * (j - i + 1)
```

ars = max (ars, area)

}

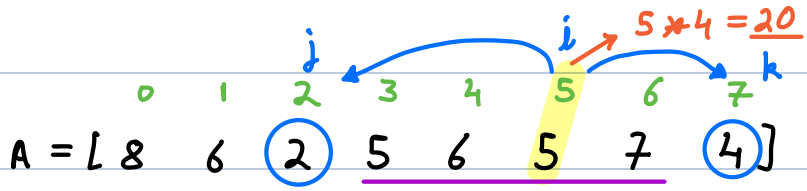
} return ars TC = O(N²) SC = O(1)

∀ width find max height i.e. min A[i—j]

Area → H * W

Sol

∀i, H = A[i] → find max width, calculate area & store max as answer.



j → index of nearest smaller on left of i

k → index of nearest smaller on right of i

[(j+1) (k-1)] ⟶ (k-1) - (j+1) + 1

= k - 1 - j - 1 + 1 = k - j - 1

TC = O(N) SC = O(N)

Sol → 1) ∀i, find & store nearest smaller on left → j

2) ∀i, find & store nearest smaller on right → k

3) ans = $\max_{\forall i} (A[i] * (k - j - 1))$

Predict Output of following code:

```java
import java.util.*;

public class StockSpanExample {

    public static int[] calculateSpan(int[] prices) {
        int n = prices.length;
        int[] span = new int[n];
        Stack<Integer> stack = new Stack<>();

        stack.push(0);
        span[0] = 1;

        for (int i = 1; i < n; i++) {

            while (!stack.isEmpty() && prices[stack.peek()] <= prices[i]) {
                stack.pop();
            }

            if (stack.isEmpty())
                span[i] = i + 1;
            else
                span[i] = i - stack.peek();

            stack.push(i);
        }

        return span;
    }

    public static void main(String[] args) {
        int[] prices = {100, 80, 60, 70, 60, 75, 85};
        int[] span = calculateSpan(prices);
    }
}
```